

Agent 框架与平台横评

语言策略：shared（Java canonical, P/T 同文）

TL;DR

- 1. 先选“控制模型”，再选框架：固定流程用 Graph，动态任务用 Agent Loop，混合用 Agentic Workflow。
- 2. 对本知识库三种语言生态，基线是：Java 用 Spring AI Alibaba Graph，Python 用 LangGraph，TypeScript 用 LangGraph.js。
- 3. 框架不是越新越好，优先看：可观测、状态持久化、人工审批、成本控制、社区与许可。

1. 选型六维度

维度	关键问题
语言适配	是否原生支持团队技术栈
控制模型	Graph / Agent Loop / Multi-Agent 支持度
状态与持久化	是否支持 checkpoint、断点恢复
可观测	Trace、回放、人工审批能力
运维	超时、重试、并发、流式、限流
生态与许可	社区活跃度、维护方、License

2. 框架对比基线

框架	语言生态	控制模型	适合场景	注意点
LangGraph	Python	Graph + Agent	复杂状态机、生产级 Agent	概念多，需团队学习
LangGraph.js	TypeScript	Graph + Agent	Node 服务端生产级 Agent	生态略小于 Python
Spring AI Alibaba Graph	Java	Graph + Agent	Java 后端、Spring 生态	按 Spring 体系选型
OpenAI Agents SDK	Python / TypeScript	Agent Loop + Handoff	快速原型、多 Agent 交接	与供应商绑定较深
AutoGen	Python / .NET	Multi-Agent	研究型多智能体	生产化需自行补运维
CrewAI	Python	Role-based Multi-Agent	内容生产流水线	复杂控制流要评估
LlamaIndex Workflows	Python	Graph + RAG	文档密集型应用	与 LangGraph 定位重叠

3. 按语言选型

技术栈	首选	备选	不建议一上来就用
Java	Spring AI Alibaba Graph	自研轻量 Agent Loop	跨语言框架拼装
Python	LangGraph	OpenAI Agents SDK	AutoGen/CrewAI 直接上生产
TypeScript	LangGraph.js	OpenAI Agents SDK	从 Python 框架翻译

4. 决策流程

1	任务能否拆成固定步骤？
2	能 → Workflow / Graph (Spring AI Alibaba Graph、LangGraph、LangGraph.js)
3	不能 → 路径是否动态？
4	动态 → Agent Loop (Agents SDK 或 Graph 内嵌 Agent)
5	需要多角色 → 先证明单 Agent 不够，再上多 Agent

5. 防锁定

- Agent 逻辑与框架 API 之间加一层 AgentRuntime / ToolRegistry 接口；
- 工具以 JSON Schema 注册，Prompt 与工具描述版本化；
- 通过 LLM Gateway 隔离模型供应商；
- 框架升级前用评测集回归，不直接追新版本。

6. 延伸阅读

- [AI 工作流中的 Workflow、Graph 与 Loop](#)
- [多智能体编排](#)
- [Agent 部署与发布](#)